

AgentCore Memory Operations Deep-Dive

Metadata Options · Streaming & Batch · Issues/Fixes by Phase Unit Testing · Evaluation Retirement · Cleanup Strategies

SECTION	CONTENTS
1 · Metadata Options	CreateEvent metadata fields, payload types, ListEvents filters, memory-record metadata, Kinesis stream event schema
2 · Streaming Use Case	Kinesis Data Stream integration, real-time change notifications, consumer patterns, event schema, IAM wiring
3 · Batch Use Case	BatchCreateMemoryRecords, BatchUpdateMemoryRecords, BatchDeleteMemoryRecords — limits, idempotency, retry patterns
4 · Issues & Fixes by Phase	PoC → Cross-session → Multi-agent → Production — known issues, root causes, fixes, CloudWatch signals
5 · Unit Testing	Mock patterns, test matrix, PII tests, isolation tests, hook tests, extraction tests, erasure tests — full code
6 · Evaluation & Retirement	Metric thresholds, retire/graduate criteria, shadow evaluation, A/B strategy switch, evaluation teardown
7 · Cleanup Strategies	Namespace purge, TTL enforcement, actor erasure, batch sweep, test teardown, cost-driven cleanup scheduler

Companion to: AWS AgentCore Memory Architecture Guide v2.0 + Gaps & Extensions Supplement (April 2026)

1. Metadata Options — Events, Records & Retrieval Filters

1.1 CreateEvent Metadata — Key-Value Pairs on Short-Term Events

Every **CreateEvent** call accepts a metadata map of string key-value pairs that travel with the event but are not part of the conversation payload. Metadata is indexed and filterable via **ListEvents** — enabling efficient event retrieval without semantic search overhead.

Parameter	Constraint	Notes
Max metadata entries per event	15 key-value pairs	Validation error if exceeded
Key length	1–128 characters	Case-sensitive; no spaces
Value length	1–256 characters	String only; no nested objects
Filter operator (ListEvents)	Exact match on key=value	No range or wildcard support on metadata
Metadata stored with	Short-term event (not extracted to long-term)	Use namespace/content for long-term recall
Visible in GetEvent response	Yes — full metadata map returned	CloudTrail logs include metadata keys only

CreateEvent with metadata — full parameter set:

```
import boto3, time, uuid
client = boto3.client('bedrock-agentcore', region_name='eu-central-1')
response = client.create_event(
    memoryId = 'mem-abcdef1234-Ab1B2c3d4e',
    actorId = actor_id, # from IdP – NEVER user-provided
    sessionId = session_id,
    clientToken= str(uuid.uuid4()), # idempotency key
    eventTimestamp = int(time.time()),
    metadata = {
        'channel': {'stringValue': 'web-chat'},
        'product_line': {'stringValue': 'wealth-management'},
        'regulatory_ctx': {'stringValue': 'mifid2'},
        'case_type': {'stringValue': 'suitability-review'},
        'pii_redacted': {'stringValue': 'true'}, # audit flag
        'consent_basis': {'stringValue': 'legitimate-interest'},
        'agent_version': {'stringValue': '2.4.1'},
    },
    payload = [{
        'conversational': {
            'content': {'text': redacted_user_message},
            'role': 'USER'
        }
    }])
```

1.2 Payload Types in CreateEvent

Payload Type	Structure Key	Use Case	Example
Conversational	conversational: {role, content: {text}}	Standard chat turn	User question / agent response
Blob / binary	blob: {mimeType, data}	PDF, image, audio stored in memory	KYC document reference
Structured JSON	structured: {content: {text}}	Tool outputs, API responses, extracted entities	Order status JSON, account details
Multi-payload (array)	Array of above	Store multiple data types in one event call	Text + metadata blob in same event

1.3 ListEvents with Metadata Filters

```
paginator = client.get_paginator('list_events')
pages = paginator.paginate(
    memoryId = memory_id,
    actorId = actor_id,
    sessionId = session_id, # optional – omit for cross-session
    metadataFilter = {
        'equals': [
            {'key': 'product_line', 'value': 'wealth-management'},
            {'key': 'regulatory_ctx', 'value': 'mifid2'}
        ]
    },
    PaginationConfig={'PageSize': 50}
)
events = [e for page in pages for e in page['events']]
```

1.4 Memory Record Metadata (Long-Term)

Long-term memory records (created by strategies or BatchCreateMemoryRecords) carry a different metadata concept: **namespace** (routing/isolation key) and **memoryRecordId** (idempotency + update anchor). These are set on write and are filterable by namespace prefix in ListMemoryRecords.

Field	Set By	Purpose	Example Value
memoryRecordId	Caller (BatchCreate) or auto-generated	Idempotency; enables BatchUpdate to overwrite	actor123-risk-appetite
namespace	Strategy config or BatchCreate caller	Routing, isolation, IAM condition key	/strategy/sem-001/actor/u-123/

content.text	Strategy extraction or BatchCreate	Actual memory content	'Risk appetite: balanced'
sourceEventIds[]	Auto-populated by strategy	Provenance — which events produced this record	[evt-001, evt-002]
createdAt / updatedAt	Service-managed	TTL decisions; staleness audit	ISO-8601 timestamps
score	Returned by Retrieve MemoryRecords only	Semantic relevance 0–1	0.91

2. Streaming Use Case — Kinesis Real-Time Change Notifications

AgentCore Memory can push real-time notifications to a **Kinesis Data Stream** whenever a memory record is created, updated, or deleted. This enables event-driven downstream architectures — compliance dashboards, fraud monitors, CRM sync, audit pipelines — without polling the list APIs.

2.1 Streaming Architecture

Layer	Component	Role
Producer	AgentCore Memory service	Publishes record-lifecycle events to Kinesis on every Create / Update / Delete
Stream	Kinesis Data Stream (in your account)	Buffer; configurable shard count and retention (1–365d)
Consumer	Lambda / Kinesis Consumer App / Firehose	Processes events in near-real-time; fan-out via EventBridge Pipes
IAM Role	bedrock-agentcore.amazonaws.com trust	Must have kinesis:PutRecord + optional kms:GenerateDataKey
Encryption	SSE via KMS (optional)	If enabled, role must also have kms:Decrypt on consumer side

2.2 Wiring — CreateMemory with Stream Delivery

```
import boto3
control = boto3.client('bedrock-agentcore-control', region_name='eu-central-1')
response = control.create_memory(
    name = 'banking-memory-streaming',
    description = 'EU Banking memory with Kinesis change stream',
    eventExpiryDuration = 90,
    encryptionKeyArn = 'arn:aws:kms:eu-central-1:123:key/cm-k-id',
    streamDeliveryResource = {
        'kinesisDataStream': {
            'streamArn': 'arn:aws:kinesis:eu-central-1:123:stream/memory-changes',
        },
        'memoryExecutionRoleArn': 'arn:aws:iam::123:role/AgentCoreStreamRole',
        'memoryStrategies': [
            {'semanticMemoryStrategy': {'name': 'FactExtractor'}}
        ]
    }
)
```

2.3 Kinesis Event Schema — Memory Record Lifecycle

Each Kinesis record DATA payload is a JSON object with the following structure:

```
{
  'eventType': 'CREATE' | 'UPDATE' | 'DELETE',
  'memoryId': 'mem-abcdef1234-Ab1B2c3d4e',
  'memoryRecordId': 'actor123-risk-appetite',
  'namespace': '/strategy/sem-001/actor/u-123/',
  'actorId': 'cognito-sub-xxxx',
  # for routing
  'eventTimestamp': '2026-04-09T10:23:45Z',
  'memoryRecord': {
    # null on DELETE
    'content': {
      'text': 'Risk appetite: balanced'
    },
    'sourceEventIds': ['evt-001', 'evt-002'],
    'createdAt': '2026-04-09T10:23:40Z',
    'updatedAt': '2026-04-09T10:23:45Z'
  },
  'previousMemoryRecord': {
    ...
  } # null on CREATE
}
```

2.4 Consumer Lambda — Compliance Audit Pipeline

```
import json, boto3
firehose = boto3.client('firehose')
def handler(event, context):
    for record in event['Records']:
        payload = json.loads(record['kinesis']['data'])
        evt_type = payload['eventType']
        actor_id = payload['actorId']
        # Route to compliance audit store if evt_type in ('CREATE', 'UPDATE') and 'pep' in payload.get('namespace', '')
        firehose.put_record(
            DeliveryStreamName='compliance-memory-audit',
            Record={
                'Data': json.dumps({
                    'event_type': evt_type,
                    'actor_id': actor_id,
                    'namespace': payload['namespace'],
                    'content': payload['memoryRecord']['content']['text'],
                    'ts': payload['eventTimestamp'],
                })
            }
        )
        # Alert on unexpected DELETE if evt_type == 'DELETE':
        # Verify this was a sanctioned SAR/erasure workflow if 'erasure-workflow' not in payload.get('namespace', '')
        if evt_type == 'DELETE':
            raise Exception(f'Unsanctioned DELETE for actor {actor_id}')
```

Streaming Issues Quick-Reference:

Issue	Symptom	Fix
Shard hot-spotting	Single Kinesis shard overwhelmed; records dropped	Set partition key = actorId[:3] (hash spread) not constant string
Out-of-order events	Consumer sees DELETE before CREATE for same record	Use sequence number from Kinesis; sort within actor_id window
Consumer falling behind	Iterator age > 1 min; memory changes missed	Increase shard count; add Enhanced Fan-Out consumer

CMK decrypt failure on consumer	AccessDeniedException in consumer Lambda	Add kms:Decrypt on consumer role for stream's CMK
Duplicate events	Same memoryRecordId processed twice	Implement idempotency table (DynamoDB) keyed on record: sequenceNumber

3. Batch Use Case — BatchCreate / Update / Delete Memory Records

The three batch APIs operate on long-term memory records directly, bypassing the async extraction pipeline. They are used by self-managed Lambda extractors, migration scripts, SAR (Subject Access Request) cleanup, and direct data hydration.

3.1 API Comparison

Operation	Endpoint	Max Records	Idempotency	Common Use
BatchCreateMemoryRecords	POST /memories/{id}/memoryRecords/ batchCreate	100 per call	clientToken per call; memoryRecordId for record-level	Self-managed extractor, initial data load, migration
BatchUpdateMemoryRecords	POST /memories/{id}/memoryRecords/ batchUpdate	100 per call	memoryRecordId must match existing record	Correct stale preferences, update entity facts post-review
BatchDeleteMemoryRecords	POST /memories/{id}/memoryRecords/ batchDelete	100 per call	Idempotent — delete non-existent returns success	GDPR erasure, TTL sweep, stale record pruning, test cleanup

3.2 BatchCreateMemoryRecords — Full Example

```
response = client.batch_create_memory_records( memoryId = memory_id, clientToken = str(uuid.uuid4()), #
call-level idempotency records = [ { 'memoryRecordId': f'{actor_id}-risk-appetite', # stable id → allows
BatchUpdate later 'content': { 'text': 'Risk appetite: balanced'}, 'namespace':
f'/actors/{actor_id}/financial-profile/' }, { 'memoryRecordId': f'{actor_id}-inv-horizon', 'content':
{ 'text': 'Investment horizon: 12 years'}, 'namespace': f'/actors/{actor_id}/financial-profile/' }, # ...
up to 100 records per call ] ) # Partial failure pattern – check per-record status for result in
response.get('results', []): if result['status'] != 'SUCCESS': logger.error(f'Failed:
{result["memoryRecordId"]} – {result["errorMessage"]}') dead_letter_queue.send(result)
```

3.3 Pagination Pattern for Batch > 100 Records

```
def batch_create_all(client, memory_id, records, batch_size=100): """Split large record lists into
batches of <=100.""" failed = [] for i in range(0, len(records), batch_size): chunk =
records[i:i+batch_size] try: resp = client.batch_create_memory_records( memoryId = memory_id,
clientToken = str(uuid.uuid4()), records = chunk ) failed.extend([ r for r in resp.get('results',[]) if
r['status']!='SUCCESS' ]) except client.exceptions.ThrottlingException: time.sleep(2 ** (i //
batch_size)) # exponential backoff # re-queue chunk to DLQ for retry failed.extend(chunk) return failed
```

3.4 Streaming vs Batch — Decision Guide

Scenario	Use Streaming (Kinesis)	Use Batch API	Notes
Real-time compliance alerting	YES	No	Alert within seconds of memory creation
CRM / downstream system sync	YES	Alternative	Streaming preferred for <5s latency
Initial data migration (10K+ records)	No	YES	Batch create in chunks; no stream needed
GDPR erasure (actor deletion)	No	YES — BatchDelete	Batch delete all records in actor namespace
Self-managed Lambda extractor	No	YES — BatchCreate	Lambda writes extracted records back to AgentCore
Monitoring memory health	YES (consume stream)	No	Count creates/deletes per hour in CloudWatch
Test environment teardown	No	YES — BatchDelete	Delete all test actor records before next run
Nightly stale preference purge	No	YES — BatchDeleteafter ListMemoryRecords	TTL-driven cleanup job

4. Issues, Root Causes & Fixes — By Development Phase

Phase 1 — PoC (Weeks 1–4)

Issue	Root Cause	Fix	CloudWatch Signal
Long-term memories never appear	No memory strategy added to CreateMemory	Add at least one strategy (SUMMARIZATION) on CreateMemory or UpdateMemory	bedrock-agentcore/memory/ExtractionJobsFailed > 0
actor_id from request body used	Developer shortcut for testing	Remove immediately — set actor_id = Cognito sub from JWT only	Audit: grep codebase for hardcoded actor
batch_size=1 (default)	Default Strands SDK behaviour	Set AgentCoreMemorySessionManager(batch_size=10)	CreateEvent calls >> expected; cost spike
PII visible in CloudWatch logs	Message content logged before PII hook fires	PIIRedactionHook must be FIRST in hooks list; add log sanitiser	Macie alert on CW log group
Memory resource in us-east-1	Default region	Force eu-central-1 in boto3 client and AgentCore config	list_memories returns ARN with wrong region
Retrieval returns 0 results in PoC	Async extraction not complete; no wait	For PoC validation, add 30s sleep after CreateEvent; production: use short-term events first	ExtractionJobsCompleted metric = 0 after 60s

Phase 2 — Cross-Session & Long-Term (Weeks 5–8)

Issue	Root Cause	Fix	CloudWatch Signal
Preferences not recalled next session	Namespace mismatch between write and retrieve	Use identical namespace template in both strategy config and RetrievalConfig	RetrieveMemoryRecords returns [] for known actors
Stale preferences returned	No TTL set on USER_PREFERENCE records	Set memoryExpiryDuration per strategy; run nightly TTL sweep Lambda	MemoryRecordAge metric > configured threshold
Consent hook not blocking non-consented actor	ConsentCheckHook not wired to real service (stubbed)	Wire ConsentCheckHook to real consent DB; test with revoked-consent actor fixture	Memory writes for non-consented actors (audit)
PII survives redaction for financial numbers	Regex misses '£85,000 annual income' style	Add custom Lambda step with financial regex: IBAN, amounts, sort codes	Macie scan finds structured PII in vector store
Cross-session actor_id collision	Test shared actor_id across test accounts	Use UUID per test actor in tests; enforce actor_id length >= 20 chars	ListEvents returns unexpected events for actor
Right-to-erasure incomplete	memory_delete deletes only one namespace	Cascade: delete events (ListEvents → DeleteEvent) + all namespace prefixes + vector records	Post-erasure: RetrieveMemoryRecords must return []

Phase 3 — Multi-Agent (Weeks 9–12)

Issue	Root Cause	Fix	CloudWatch Signal
Sub-agent writes to wrong namespace	All agents share same IAM role without condition	IAM Condition: StringLike bedrock-agentcore:namespace = 'agent-X/*' per role	Unexpected namespace entries in ListMemoryRecords
Race condition on shared namespace	Two sub-agents write simultaneously; last-write-wins corrupts data	Hub & Spoke: sub-agents write to own namespace; orchestrator consolidates	Duplicate or missing facts in shared namespace
Orchestrator sees stale sub-agent output	Async consolidation lag; orchestrator reads before sub-agents finish	Orchestrator polls sub-agent namespace completion before consolidating	ExtractionJobsInProgress > 0 when orchestrator reads
AgentCore Policy Cedar rule too broad	Wildcard in resource: '*' instead of specific memory ARN	Scope all Cedar policies to specific memory ARN and namespace prefix	CloudTrail: unexpected cross-namespace reads

Issue	Root Cause	Fix	CloudWatch Signal
Memory injection adds >1000 tokens per turn	All 5 retrieval tiers injected regardless of relevance	Implement tiered budget (900 token cap); skip tier 5 if no episodic	Input token count spike in BedrockRuntime metrics
Transaction ledger missing HMAC on old events	CheckpointHook added after go-live of ledger	Backfill HMAC via BatchUpdateMemoryRecords with signed content field	Compliance audit finds events without signature field

Phase 4 — Production (Weeks 13+)

Issue	Root Cause	Fix	CloudWatch Signal
Memory retrieval p99 > 500ms	No retrieval cache; vector search on every turn	Add 15-min retrieval cache keyed on actor_id; warm cache on session start	RetrieveMemoryRecords duration P99 in CW
Episodic memory shows demographic bias	Reflection agent trained on biased historical sessions	Run quarterly fairness audit; disable episodic if delta > 10% across segments	EpisodeBiasDelta metric > 0.05
Kinesis stream lagging	Shard count undersized for concurrent users	Scale shards = ceil(write_TPS / 1000); enable Enhanced Fan-Out	GetRecords.IteratorAgeMilliseconds > 60000
BatchDelete fails on partial records	Missing memoryRecordId for some records	Paginate ListMemoryRecords first; collect all IDs; batch delete in chunks of 100	BatchDeleteMemoryRecords 4xx errors
CMK rotation breaks existing sessions	KMS key rotated; old ciphertext fails on decrypt	AWS auto-rotation preserves old key versions for decrypt; ensure enable_key_rotation=true	DecryptionFailures in CMK CloudWatch metric
Cost overrun from batch_size=1 in legacy code	Old code path not updated after SDK migration	Audit all Agent() constructors; enforce batch_size>=10 in CI linter	CreateEvent calls > 10x expected rate
GDPR Art.22 automated decision without review path	Episodic memory influences credit decision with no human override	Add HumanReviewRequired flag to episodic memory namespace; block auto-approval if set	Compliance: decision logs without review_flag field

5. Unit Testing — Mocks, Test Matrix & Full Test Code

All AgentCore Memory unit tests **MUST** run against mocked clients. Never call real AgentCore APIs in unit tests — they create billable events and leave orphaned test data. Use **unittest.mock.patch** or **moto** (where supported) for all boto3 clients.

5.1 Mock Architecture

```
import unittest from unittest.mock import MagicMock, patch, call import pytest # Fixture: shared mock
client @pytest.fixture def mock_agentcore(): with patch('boto3.client') as mock_boto: client =
MagicMock() mock_boto.return_value = client # Pre-configure default happy-path responses
client.create_event.return_value = { 'event': { 'eventId': 'evt-001', 'actorId': 'test-actor-uuid' } }
client.retrieve_memory_records.return_value = { 'memoryRecordSummaries': [ { 'content': { 'text': 'Risk
appetite: balanced'}, 'score': 0.92, 'memoryRecordId': 'test-actor-uuid-risk', 'namespace':
'/actors/test-actor-uuid/' } ] } client.batch_create_memory_records.return_value = { 'results':
[{'memoryRecordId': 'r1', 'status': 'SUCCESS'}] } client.batch_delete_memory_records.return_value =
{'results': []} yield client
```

5.2 Test Matrix — Complete Coverage

Test ID	What Is Tested	Pass Criterion	Fail Action
UT-MEM-001	PII redaction fires before CreateEvent	Zero PII patterns (IBAN, name, email) in create_event call args	Block deployment; page security team
UT-MEM-002	actor_id comes from mock IdP, not request body	create_event called with actor_id == cognito_sub fixture, not 'user_input'	Block deployment
UT-MEM-003	batch_size >= 10 enforced	After 10 messages: create_event called exactly once (batch flush)	Fail with cost-risk annotation
UT-MEM-004	Memory retrieval injected into system prompt	system_prompt contains mock memory text after on_message_added fires	Test failure; retrieval hook broken
UT-MEM-005	Consent check disables memory for no-consent actor	No create_event calls when has_valid_basis returns False	GDPR violation if fails
UT-MEM-006	Cross-tenant isolation: actor A cannot read actor B	retrieve_memory_records for actor B raises AuthError with actor A credentials	CRITICAL GDPR breach if fails
UT-MEM-007	Right-to-erasure cascades all namespaces	After memory_delete(actor_id): retrieve_memory_records returns []	GDPR Art.17 breach if fails
UT-MEM-008	BatchCreate partial failure routes to DLQ	Mock 1 failed + 1 success in batch result → DLQ receives failed record	Data loss if fails
UT-MEM-009	Kinesis consumer handles out-of-order events	DELETE before CREATE for same ID → idempotency table prevents double-delete	Corrupted memory state if fails
UT-MEM-010	Metadata filter returns only matching events	ListEvents with metadata filter returns only events with matching key=value	Wrong event retrieval
UT-MEM-011	Summarising hook does not double-summarise with AgentCore SUMMARIZATION	Mock verifies summarize() not called when AgentCore strategy active	Redundant cost
UT-MEM-012	Checkpointing fires after tool-heavy turn	checkpoint.save called when event.had_tool_calls = True	Lost checkpoint on failure
UT-MEM-013	Structured extraction produces valid Pydantic model	Lambda extractor parses mock LLM response into FinancialProfile without ValidationError	Extraction failure
UT-MEM-014	memory_delete requires DPO-admin role; writer role raises 403	BatchDeleteMemoryRecords raises AccessDeniedException for writer IAM role mock	GDPR access control failure

5.3 PII Redaction Test — Zero Tolerance

```
import re from tests.fixtures import PII_PATTERNS # IBAN, card, NIN, email, name, DOB
PII_PATTERNS = [
    r'[A-Z]{2}[0-9]{2}[A-Z0-9]{4}[0-9]{7}([A-Z0-9?){0,16}', # IBAN
    r'[0-9]{4}[- ][0-9]{4}[- ][0-9]{4}[- ][0-9]{4}', # card
    r'[A-Z]{2}[0-9]{6}[A-Z]', # NIN
    r'[a-zA-Z0-9._%+\-]+@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}', # email
]
def test_pii_zero_tolerance(mock_agentcore):
    hook = PIIRedactionHook(redactor=PIIRedactor())
    test_message = 'My IBAN is GB33BUKB20201555555555, card 4111-1111-1111-1111'
    event = FakeMessageEvent(content=test_message, role='user')
    result = hook.on_message_added(event)
    stored_content = result.message.content
    for pattern in PII_PATTERNS:
        matches = re.findall(pattern, stored_content)
    assert matches == [], (f'PII SURVIVED REDACTION - pattern {pattern} found: {matches}')
    # Verify the event was still written (not silently dropped)
    assert '[REDACTED]' in stored_content or 'XXXXXXXX' in stored_content
```

5.4 Isolation Test — Cross-Tenant Barrier

```
def test_actor_isolation(mock_agentcore):
    actor_a = 'cognito-sub-actor-a-uuid'
    actor_b = 'cognito-sub-actor-b-uuid'
    # Write memory for actor A
    mock_agentcore.batch_create_memory_records(
        memoryId='test-memory',
        records=[{'memoryRecordId': 'a-pref-1', 'content': {'text': 'Actor A secret preference'}},
        {'namespace': f'/actors/{actor_a}/'}])
    # Attempt retrieval using actor B's namespace - must return EMPTY
    mock_agentcore.retrieve_memory_records.return_value = {'memoryRecordSummaries': []}
    # IAM condition enforced by mock
    result = mock_agentcore.retrieve_memory_records(
        memoryId='test-memory',
        namespace=f'/actors/{actor_b}/',
        searchQuery='secret preference')
    assert result['memoryRecordSummaries'] == [], \
        'CRITICAL: Actor B retrieved Actor A memories - isolation FAILED'
```

5.5 Erasure Completeness Test

```
def test_right_to_erasure_completeness(mock_agentcore):
    actor_id = 'test-erasure-actor-uuid'
    namespaces = [
        f'/actors/{actor_id}/preferences/',
        f'/actors/{actor_id}/financial-profile/',
        f'/strategy/sem-001/actor/{actor_id}/',
    ]
    # Invoke erasure skill
    memory_delete_skill(actor_id=actor_id, memory_id='test-memory')
    # Verify ALL namespaces return empty after deletion
    for ns in namespaces:
        result = mock_agentcore.retrieve_memory_records(
            memoryId='test-memory',
            namespace=ns,
            searchQuery='any')
        assert result['memoryRecordSummaries'] == [], \
            f'GDPR BREACH: Records remain in {ns} after erasure'
    # Verify events also deleted
    mock_agentcore.list_events.return_value = {'events': []}
    events = mock_agentcore.list_events(
        memoryId='test-memory',
        actorId=actor_id)
    assert events['events'] == [], 'Events survived erasure - GDPR Art.17 violation'
```

6. Evaluation Metrics, Retirement Criteria & Strategy Switching

6.1 Full Metric Definitions & Retirement Thresholds

Metric	Definition	Target	Graduate Threshold	Retire/Rollback Threshold	Alert Level
Memory Retrieval Relevance	Cosine similarity: retrieved memory vs current query (sampled 10% of turns)	≥0.85	≥0.90 sustained 7 days	<0.70 sustained 24h	P1
PII Leakage Rate	% of create_event calls where Macie/Presidio detects unredacted PII in content field	0.00%	0.00% always	Any single PII event	P0 — page DPO within 1h
Cross-Session Recall	% of sessions where agent correctly uses preference from prior session (LLM-as-judge on sampled 5%)	≥90%	≥95% sustained 14 days	<75% sustained 48h	P2
Memory Staleness Rate	% of retrieved records older than namespace TTL config	<5%	<2%	>15% sustained 24h	P2 — run TTL purge
Namespace Isolation	% of cross-namespace read attempts blocked by IAM	100%	100% always	Any bypass	P0 — suspend agent
Preference Extraction Precision	% of extracted USER_PREFERENCE records matching stated preference (human spot-check 20 records/week)	≥85%	≥90%	<70% two consecutive weeks	P2
Erasure Completeness	After SAR erasure: 0 records retrievable for actor_id (automated post-erasure probe)	100%	100% always	Any record surviving erasure	P0 — GDPR breach
Episodic Bias Score	Max demographic fairness delta in episodic-influenced decisions (quarterly audit)	Δ<5%	Δ<2%	Δ>10% (suspend episodic immediately)	P1
Retrieval Latency p99	End-to-end: RetrieveMemoryRecords + inject + model first token	<500ms	<200ms	≥1s sustained 15min	P2
Batch Success Rate	% of BatchCreate/Update/Delete records with status=SUCCESS	≥99.9%	100%	<99% sustained 1h	P2
Extraction Job Success Rate	% of consolidation jobs completing without ExtractionJobsFailed metric increment	≥99%	100%	<95% sustained 2h	P1
Token Budget Compliance	% of turns where injected memory tokens ≤ 900 token budget	≥95%	100%	<80% sustained 24h	P2 — tune retrieval tiers

6.2 Graduate / Retire / Rollback Decision Logic

```
def evaluate_strategy_health(metrics: dict) -> str: """ Returns: 'GRADUATE' | 'HOLD' | 'ROLLBACK' | 'SUSPEND' """ # P0 conditions – immediate action if metrics['pii_leakage_rate'] > 0: return 'SUSPEND' # disable memory writes; page DPO if metrics['namespace_isolation'] < 1.0: return 'SUSPEND' # cross-tenant leak; disable all memory reads if metrics['erasure_completeness'] < 1.0: return 'SUSPEND' # GDPR Art.17 breach; DPO notification required # P1 conditions – rollback strategy if metrics['retrieval_relevance_7d'] < 0.70: return 'ROLLBACK' # switch back to previous strategy config if metrics['episodic_bias_delta'] > 0.10: return 'ROLLBACK' # disable episodic; EBA ML audit required # Graduate conditions – all green if (metrics['retrieval_relevance_7d'] >= 0.90 and metrics['recall_14d'] >= 0.95 and metrics['pii_leakage_rate'] == 0 and metrics['latency_p99_ms'] < 200): return 'GRADUATE'
```

```
return 'HOLD'
```

6.3 Shadow Evaluation — A/B Strategy Switch

When switching extraction strategies (e.g. built-in SEMANTIC → self-managed Lambda), run shadow evaluation for 7 days before cutting over:

Step	Action	Duration	Success Criterion
1 — Dual write	New strategy added alongside existing; both extract from same events	7 days	New strategy extraction rate > 0; no extraction failures
2 — Sample compare	Human review: sample 50 records from each strategy; precision scored	7 days	New strategy precision ≥ existing ± 5%
3 — Retrieval A/B	10% of retrieval calls use new strategy namespace; relevance scored	7 days	New strategy retrieval relevance ≥ existing - 0.02
4 — Cutover	Set new strategy as primary; old strategy set to read-only	Day 15	Latency + relevance hold; no new PII events
5 — Drain old records	Run TTL sweep on old strategy namespace; BatchDelete orphans	Day 21	Old namespace empty; storage cost reduced

6.4 Evaluation Teardown — Retiring a Memory Strategy

```
def retire_strategy(client, memory_id: str, strategy_id: str, actor_ids: list): """ Safely retire a memory strategy: 1. Drain all records in strategy namespace 2. Remove strategy from memory resource 3. Verify namespace is empty """ namespace_prefix = f'/strategy/{strategy_id}/' for actor_id in actor_ids: ns = f'{namespace_prefix}actor/{actor_id}/' # Paginate all records in this namespace record_ids = [] paginator = client.get_paginator('list_memory_records') for page in paginator.paginate(memoryId=memory_id, namespace=ns): record_ids.extend([ r['memoryRecordId'] for r in page['memoryRecordSummaries'] ]) # BatchDelete in chunks of 100 for i in range(0, len(record_ids), 100): client.batch_delete_memory_records( memoryId=memory_id, records=[{'memoryRecordId': rid} for rid in record_ids[i:i+100]] ) # Remove strategy from memory resource config client.update_memory( memoryId=memory_id, memoryStrategies=[ {'remove': {'memoryStrategyId': strategy_id}} ] ) # Verify remaining = client.list_memory_records( memoryId=memory_id, namespace=namespace_prefix ) assert not remaining['memoryRecordSummaries'], \ f'Strategy retirement incomplete – records remain in {namespace_prefix}'
```

7. Cleanup Strategies — Namespace Purge, TTL, Erasure & Scheduler

7.1 Cleanup Taxonomy

Cleanup Type	Trigger	Scope	API Used	Frequency
Session-end event purge	EventBridge 'Session Ended'	Events older than short-term TTL	DeleteEvent (per event) or auto-expire	Managed by retention setting
Actor right-to-erasure	SAR request (GDPR Art.17)	ALL events + ALL memory records for actor_id	DeleteEvent + Batch DeleteMemoryRecords	On-demand; < 30 days SLA
Namespace TTL sweep	Scheduled Lambda (EventBridge Scheduler)	Records where createdAt > TTL threshold	ListMemoryRecords → BatchDeleteMemoryRecords	Nightly
Stale preference purge	Records older than USER_PREFERENCE TTL config	USER_PREFERENCE namespace per actor	BatchDeleteMemoryRecords	Weekly
Strategy retirement drain	Strategy removal workflow	All records in strategy namespace prefix	BatchDeleteMemoryRecords	One-time; see §6.4
Test environment teardown	CI/CD pipeline post-test hook	All records under /test/ namespace prefix	BatchDeleteMemoryRecords	Per test run
Full memory resource deletion	Project shutdown / environment destroy	All events + records in memory resource	DeleteMemory (irreversible)	One-time; requires DPO approval for production
Duplicate record deduplication	Consolidation job or scheduled Lambda	Records with identical content in same namespace	BatchDeleteMemoryRecords (keep latest)	Weekly / post-migration

7.2 Actor Right-to-Erasure — Production-Grade Implementation

```
def full_actor_erasure( client, memory_id: str, actor_id: str, strategy_ids: list, dry_run: bool = True ) -> dict: """ GDPR Art.17 compliant erasure. dry_run=True audits without deleting. Returns: {events_deleted, records_deleted, namespaces_cleared} """ audit = {'events_deleted': 0, 'records_deleted': 0, 'namespaces_cleared': []} # ███ Step 1: Delete all short-term events
████████████████████████████████████████ paginator = client.get_paginators('list_events') for page in paginator.paginate(memoryId=memory_id, actorId=actor_id): for event in page['events']: if not dry_run: client.delete_event( memoryId=memory_id, actorId=actor_id, eventId=event['eventId'] ) audit['events_deleted'] += 1 # ███ Step 2: Delete all long-term records across all strategy ns ███ namespaces = [ f'/{strategy}/{sid}/actor/{actor_id}/' for sid in strategy_ids ] + [f'/actors/{actor_id}/', f'/users/{actor_id}/'] # custom namespaces for ns in namespaces: ids_to_delete = [] paginator = client.get_paginators('list_memory_records') for page in paginator.paginate(memoryId=memory_id, namespace=ns): ids_to_delete.extend([ r['memoryRecordId'] for r in page['memoryRecordSummaries'] ]) if ids_to_delete and not dry_run: for i in range(0, len(ids_to_delete), 100): client.batch_delete_memory_records( memoryId=memory_id, records=[{'memoryRecordId': rid} for rid in ids_to_delete[i:i+100]] ) audit['records_deleted'] += len(ids_to_delete) if ids_to_delete: audit['namespaces_cleared'].append(ns) # ███ Step 3: Post-erasure verification probe
████████████████████████████████████████ if not dry_run: for ns in audit['namespaces_cleared']: probe = client.retrieve_memory_records( memoryId=memory_id, namespace=ns, searchQuery='any' ) if probe['memoryRecordSummaries']: raise GPDRBreach(f'Records persist in {ns} after erasure') return audit
```

7.3 Nightly TTL Sweep Lambda

```
from datetime import datetime, timedelta, timezone def ttl_sweep_handler(event, context): """
EventBridge Scheduler → nightly at 02:00 UTC. Deletes records older than per-namespace TTL config. """
TTL_CONFIG = { '/actors/{actor_id}/preferences/': timedelta(days=365),
'/actors/{actor_id}/financial-profile/': timedelta(days=730), '/strategy/*/actor/*/session/':
timedelta(days=30), } client = boto3.client('bedrock-agentcore') now = datetime.now(timezone.utc)
deleted = 0 for ns_template, ttl in TTL_CONFIG.items(): cutoff = now - ttl stale_ids = [] paginator =
client.get_paginator('list_memory_records') for page in paginator.paginate( memoryId=MEMORY_ID,
namespace=ns_template.replace('{actor_id}', '')): for rec in page['memoryRecordSummaries']: updated =
```

```
datetime.fromisoformat(rec['updatedAt']) if updated < cutoff: stale_ids.append(rec['memoryRecordId'])
for i in range(0, len(stale_ids), 100): client.batch_delete_memory_records( memoryId=MEMORY_ID,
records=[{'memoryRecordId': rid} for rid in stale_ids[i:i+100]] ) deleted += len(stale_ids[i:i+100])
print(f'TTL sweep complete: {deleted} stale records deleted') cloudwatch.put_metric_data(
Namespace='AgentCore/Memory',
MetricData=[{'MetricName': 'TTLSweepDeleted', 'Value':deleted, 'Unit': 'Count'}] )
```

7.4 CI/CD Test Teardown Pattern

```
def pytest_sessionfinish(session, exitstatus): """pytest plugin hook – runs after all tests regardless
of pass/fail.""" client = boto3.client('bedrock-agentcore', region_name='eu-central-1') TEST_NS_PREFIX =
'/test/' # Find and delete all test records paginator = client.get_paginator('list_memory_records')
test_ids = [] for page in paginator.paginate( memoryId=os.environ['TEST_MEMORY_ID'],
namespace=TEST_NS_PREFIX): test_ids.extend([r['memoryRecordId'] for r in page['memoryRecordSummaries']])
for i in range(0, len(test_ids), 100): client.batch_delete_memory_records(
memoryId=os.environ['TEST_MEMORY_ID'], records=[{'memoryRecordId': rid} for rid in test_ids[i:i+100]] )
print(f'[teardown] Deleted {len(test_ids)} test memory records')
```

7.5 Cleanup Decision Matrix

Scenario	Best Cleanup Strategy	Risk if Skipped
Actor requests data deletion (GDPR)	full_actor_erasure() — all namespaces + events	Regulatory breach; ICO fine; reputational damage
Strategy retired or replaced	retire_strategy() + namespace drain	Orphaned records inflate storage cost; stale data retrieved
Preference data > 2 years old	Nightly TTL sweep with per-namespace TTL config	MiFID II suitability based on stale preferences
Test run completes in CI	pytest_sessionfinish teardown hook	Test actor IDs pollute production-shared dev memory
Development environment shutdown	DeleteMemory (full resource delete) — requires DPO sign-off	Ongoing cost; potential PII retention beyond purpose
Duplicate records detected post-migration	BatchDeleteMemoryRecords on duplicates; keep latest updatedAt	Conflicting facts retrieved; poor agent quality
AML/KYC memory after 7-year retention end	S3 WORM expiry + BatchDeleteMemoryRecords from AgentCore	Legal obligation to delete; ongoing storage cost
Session-scoped memories after 90 days	Automatic via eventExpiryDuration setting on Memory Resource	Short-term events accumulate; minor cost growth

Production Reference · April 2026 · Companion to AgentCore Memory Architecture Guide v2.0 + Gaps & Extensions Supplement. All code examples use mock clients in unit test context and boto3 in production context. Regulatory citations follow GDPR (2018), DORA (2025), MiFID II, EBA ML Guidelines.